

A Review of Activation Function for Artificial Neural Network

1st Andrinandrasana David Rasamoelina
dep. Cybernetics and Artificial Intelligence
Technical University of Kosice
 Kosice, Slovak Republic
 andrijdavid@tuke.sk

2nd Fouzia Adjailia
dep. Cybernetics and Artificial Intelligence
Technical University of Kosice
 Kosice, Slovak Republic
 fouzia.adjailia@tuke.sk

3rd Peter Sinčák
dep. Cybernetics and Artificial Intelligence
Technical University of Kosice
 Kosice, Slovak Republic
 peter.sincak@tuke.sk

Abstract—The activation function plays an important role in the training and the performance of an Artificial Neural Network. They provide the necessary non-linear properties to any Artificial Neural Network. In this work, we provide a review of the most common and recent activation functions used in the hidden layer of an artificial Neural Network.

Index Terms—Activation Function, Hidden Unit, Neural Network.

I. INTRODUCTION

The activation function plays an important role in the training of neural networks. They provide the necessary non-linearity of the model to be able to learn complex representations.

While different activation functions such as Sigmoid (eq. 5) or Hyperbolic Tangent (eq. 6) are often equivalent on a theoretical level. That is in the sense that they can all approximate arbitrary continuous functions [1] [2] [3] [4], different activation functions often uncover different behavior in practice. Indeed, in the early era of the neural network, Sigmoid (eq. 5) was the most used activation function in a neural network. But after several years of practice, the seldom accepted that its small derivative may lead to vanishing gradient problem. Much more suitable was ReLU (eq. 7) [5], which has a derivative of one for every positive input. Consequently, it has become the most used activation function for the neural network. The ReLU success led to enormous amount of work have been done in order to find either by human means [6] [7] [8] [9] or by automated means [10] an activation function that perform as well or better than ReLU. Unfortunately, none of them has yet reached the same popularity as ReLU mostly due to ReLU's simplicity.

In this work, we provide a simple review of the current most promising activation function that improves upon ReLU or point to a new methodology that can be applied in order to use older activation function.

II. METHODOLOGY OF COMPARISON

In this work we consider eight major activation functions, two of which are “novel” and proposed in [10] and in [11]. A brief summary of those activation function is given in the table II. It is difficult to explain why another activation function performs better than another one. Therefore we resume in table I the properties suspected to be crucial for successful learning. For every activation function, we compute the output landscape from a 4 layers neural network initialized with Kaiming Uniform [6]. It has been shown in [10] [11] that the smoothness of the output landscape results in smoother loss functions which are easier to optimize and hence the network is able to generalize better.

III. TYPES OF ACTIVATION FUNCTION

Commonly used activation function can be classified into two main categories of function: Piecewise Linear Function and Locally Quadratic Function. In this section, we briefly explain those categories and give an example for each type.

A. Piecewise Linear Activation Functions

A piecewise linear function is a function composed of a limited number of linear segments defined over an equal number of intervals, usually of equal size.

Examples of piecewise linear activation function:

$$\text{ReLU}(x) = \max(0, x) \quad (1)$$

Indeed most of the activation function from the ReLU variant can be considered as a piecewise linear function. The main characteristic of these types of activation functions is the absence of curvature in every interval defined by its breakpoint. I.e having a constant first-order derivative.

B. Locally Quadratic Activation Functions

Any non linear smooth activation function with nonzero second derivative, is locally quadratic.

Examples of locally quadratic activation function:

TABLE I
FREQUENT PROPERTIES OF ACTIVATION FUNCTIONS

Property	Description	Problems	Examples
derivative	f'	> 1 exploding gradient (e) < 1 vanishing (v)	Sigmoid, Hyperbolic Tangent
zero-centered	range centered around zero?	if not, slower learning	Hyperbolic Tangent
saturating	finite limits	vanishing gradient in the limit	Hyperbolic Tangent, ReLU
monotonicity	$x > y \implies f(x) \geq f(y)$	unclear	exceptions: Swish

$$\text{Swish}(x) = \frac{x}{1 + e^{-x}} \quad (2)$$

$$\text{Mish}(x) = x \tanh(\log(1 + e^x)) \quad (3)$$

A piecewise linear function having a non zero second derivative at least on one interval is also a locally quadratic function.

$$\text{ELU}(x) = \max(0, x) + \min(0, \alpha(e^x - 1)) \quad (4)$$

This behaviour make the approximation of the square function efficiently by any neural network using this types of activation.

IV. ACTIVATION FUNCTION

A. Sigmoid

Sigmoid activation function (eq. 5) translates the input range from $(-\infty; +\infty)$ to the range in $[0; 1]$. It is non-linear by nature and has a smooth derivative as shown in figure 1. Due to the output range of the sigmoid $[0; 1]$ the output of each unit is also squashed causing the gradient to vanish especially in a deep network. This behavior makes the network very hard to optimize and after a certain point

The Sigmoid or Logistic Activation Function function is defined as follows:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (5)$$

The output landscape of the sigmoid function is shown in the figure 2.

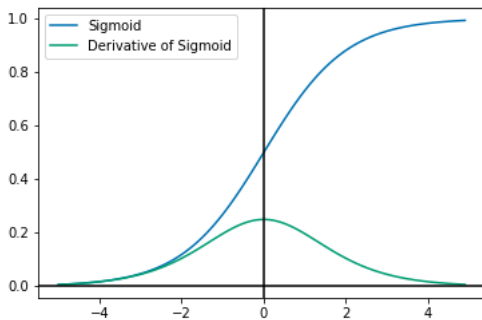


Fig. 1. Plot of the sigmoid function and its derivative.

Various work have been done into using sigmoid in artificial neural networks [14] [4] [2] [15]. In [14] the author propose the use of orthogonal weight initialization for deep sigmoidal network and proves that it can learn a magnitude of time faster than deep network using ReLU .

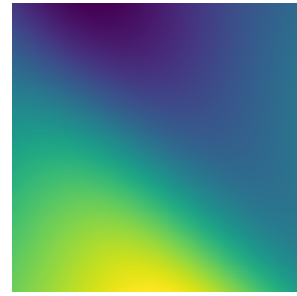


Fig. 2. Output landscape of sigmoid.

B. Hyperbolic Tangent

The Hyperbolic Tangent function has a structure very similar to Sigmoid function. However, the function squash the input value in the range of $[-1, +1]$. The main advantage over the sigmoid function is that its derivative is steeper as shown in figure 4, which means it can get more value. Despite having a steeper gradient and due to its bounded output value the gradient tends to vanish with this function. This function can be defined as follows:

$$\tanh(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (6)$$

The output landscape of the Hyperbolic Tangent function is shown in the figure 3.

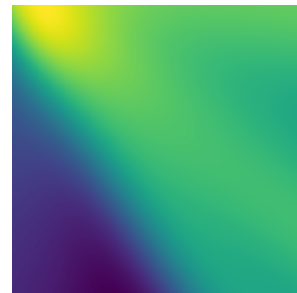


Fig. 3. Output landscape of tanh.

In order to overcome those drawbacks, [9] Swalpa Kumar et al. linearly scale the hyperbolic tangent function and achieves faster convergence and higher performance over ReLUIV-C.

TABLE II
SUMMARY OF ACTIVATION FUNCTION

Name	Equation	Derivative	Range	Order of Conti-nuity	Monotonic derivative
Sigmoid	$\frac{1}{1+e^{-x}}$	$\frac{e^{-x}}{(1+e^{-x})^2}$	$(-\infty, +\infty)$	C^∞	Yes
Hyperbolic Tan-gent	$\frac{2}{1+e^{-2x}} - 1$	$\frac{1}{\cosh^2 x}$	$(-1, 1)$	C^∞	Yes
ReLU [6] [5]	$\max(0, x)$	$\begin{cases} 0, & \text{if } x \leq 0 \\ 1, & \text{if } x > 0 \end{cases}$	$[0, \infty)$	C^0	Yes
Leaky ReLU [6]	$\max(0, x) + \alpha \min(0, x)$	$\begin{cases} 1, & \text{if } x \geq 0 \\ \alpha, & \text{otherwise} \end{cases}$	$(-\infty, +\infty)$	C^0	Yes
PReLU [6]	$\max(0, x) + \alpha \times \min(0, x)$	$\begin{cases} 1, & \text{if } x \geq 0 \\ \alpha, & \text{otherwise} \end{cases}$	$(-\infty, +\infty)$	C^0	Yes
ReLU6 [12]	$\min(\max(0, x), 6)$	$\begin{cases} 0, & \text{if } x < 0 \text{ or } x > 6 \\ 1, & \text{if } 6 \geq x \geq 0 \end{cases}$	$[0, 6)$	C^0	Yes
ELU [7]	$\max(0, x) + \min(0, \alpha(e^x - 1))$	$\begin{cases} \alpha e^x, & \text{if } x < 0 \\ 1, & \text{if } x \geq 0 \end{cases}$	$(-\alpha, +\infty)$	$\begin{cases} C^1, & \text{if } \alpha = 1 \\ C^0, & \text{if } \alpha \neq 1 \end{cases}$	Yes if $\alpha \geq 1$
SELU [13] ¹	$\gamma(\max(0, x) + \min(0, \alpha(e^x - 1)))$	$\gamma \begin{cases} \alpha e^x, & \text{if } x < 0 \\ 1, & \text{if } x \geq 0 \end{cases}$	$(-\gamma\alpha, +\infty)$	C^0	Yes
Swish [10]	$\frac{x}{1+e^{-x}}$	$\frac{e^x(x+e^x+1)}{(e^x+1)^2}$	$(\approx -0.2784, +\infty)$	C^∞	No
Mish [11]	$x \tanh(\log(1 + e^x))$	$\frac{e^x(4e^x x + 4x + 6e^x + 4e^{2x} + e^{3x} + 4)}{(2e^x + e^{2x} + 2)^2}$	$(\approx -0.3088, +\infty)$	C^∞	No

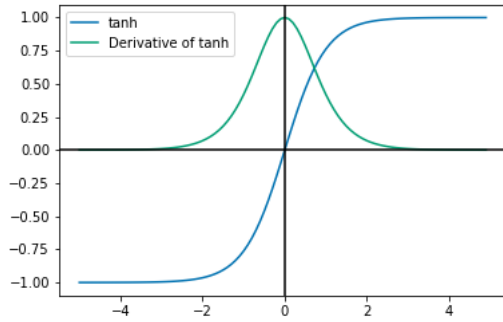


Fig. 4. Plot of the hyperbolic tangent function and its derivative.

C. Rectified Linear Unit (ReLU)

ReLU is the most used activation function among practitioner and deep learning researcher. The success of ReLU is based on its superior training performance [5] [6] over other activation functions such as the logistic sigmoid and the hyperbolic tangent.

The ReLU(Rectified Linear Units) function is defined as follows:

$$\text{ReLU}(x) = \max(0, x) \quad (7)$$

ReLU is linear (identity) for all positive values, and zero for all negative values. However, the output values range from 0 to infinity. One of the advantages of using ReLU is its simplicity. It is capable of outputting a true zero value allowing the

activation of hidden layers in neural networks to contain one or more true zero values increasing the representational sparsity of the network. ReLU is generally not generally advisable in Recurrent Neural Network because they can have very large outputs so they might be expected to be far more likely to explode than activation function that has bounded values. The major downside of ReLU is that all negative values are transformed to zero causing every negative unit to output zero. This problem is called "dying ReLU". It is a special case of the vanishing gradient problem. So once the neuron gets negative it is unlikely for it to recover [8] [16]. The output landscape of the ReLU function is shown in figure 5 and the figure 6 show the plot of ReLU and its derivative.

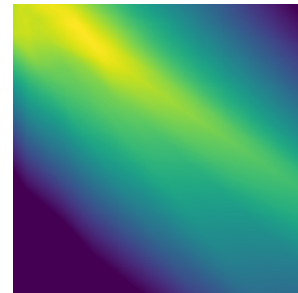


Fig. 5. Output landscape of ReLU.

In the literature multiple analysis have been done, in order to understand and improve ReLU [17] [18] [16] [19] [20] [21] [22] [23] [24] [25] [26]. Multiple methods have been proposed in order to avoid the dying ReLU problem. Lu et al. [17] proposes the uses of a randomized asymmetric weight

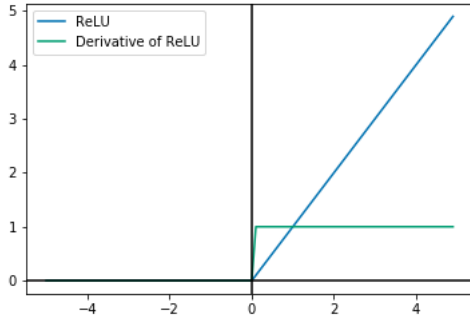


Fig. 6. Plot of the hyperbolic tangent function and its derivative.

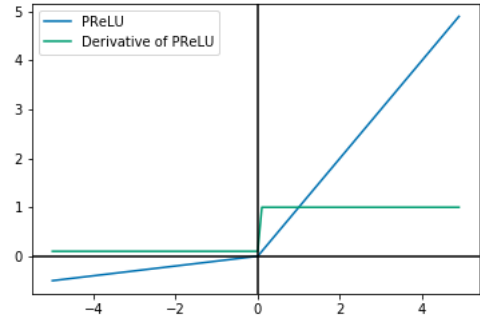


Fig. 8. Plot of the PReLU function and its derivative.

initialization. Another method by He et al. [6] is the use of a learnable parameter as seen in Section IV-D.

D. Parametric leaky version of a Rectified Linear Unit

PReLU [6] is another attempt to fix the “dying ReLU” problem by giving a negative slope α to the ReLU function. Instead of the function being zero when $x \leq 0$, it gives a small negative slope. Here α is a learnable parameter. If α is a constant then it’s called Leaky ReLU. The PReLU function can be expressed as follows:

$$\text{PReLU}(x) = \max(0, x) + \alpha * \min(0, x) \quad (8)$$

This function allows the negative slope to be learned—unlike leaky ReLU. Therefore it is possible to perform back propagation and learn the most appropriate value α .

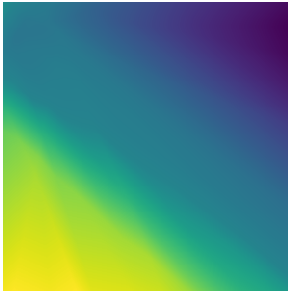


Fig. 7. Output landscape of PReLU.

E. Exponential Linear Unit

ELU was first proposed in [7] as an improvement over ReLU. It has been shown that it leads to higher classification results than traditional ReLU [8] [27] [28] [7]. For $x \geq 0$, It follows the same rule as ReLU, and for $x < 0$ it increases exponentially. As shown in the figure 10, ELU becomes smooth slowly until its output equal -1 . The main improvement of ELU over ReLU is that it is able to output negative value.

The ELU activation function can be expressed as follows:

$$\text{ELU}(x) = \max(0, x) + \min(0, \alpha(e^x - 1)) \quad (9)$$



Fig. 9. Output landscape of ELU.

In order to improve the result of ELU [8] [29] propose to use a learning parameterization of ELU in order to learn the proper activation shape at each layer in the network.

F. Scaled Exponential Linear Unit

SELU was first proposed in [13] to be used with Self-Normalized Neural Network. The SELU activation function can be expressed as follows:

$$\text{SELU}(x) = \gamma * (\max(0, x) + \min(0, \alpha(e^x - 1))) \quad (10)$$

with $\alpha = 1.6732632423543772848170429916717$ and $\gamma = 1.0507009873554804934193349852946$. This function induces self-normalizing properties. The author shows that activation close to zero mean and unit variance that are propagated through many network layers will converge towards zero mean and unit variance — even under the presence of noise and perturbations. This activation function works well in standard feed-forward neural networks(FNNs). However, for another task, the value of α and γ needs to be recomputed following the algorithm in the appendix of [13]. Although [30] used SELU in Convolutional Neural Network, little study has been done in on the usage of SELU on CNN. The shape and the output landscape of the SELU function are shown in figure 12 and 11.

G. Swish

Swish is a self-gated activation function discovered using reinforcement learning [10]. As shown in figure 14 the Swish function doesn’t have any upper bound. Unlike ReLU, Swish

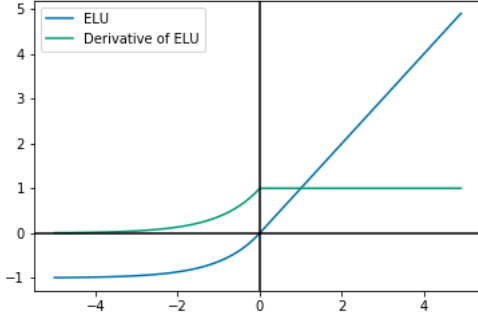


Fig. 10. Plot of the ELU function and its derivative.

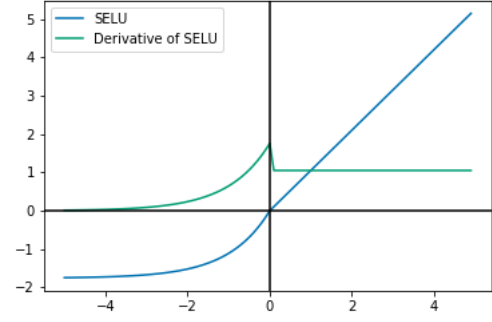


Fig. 12. Plot of the SELU function and its derivative.

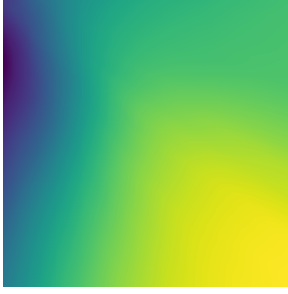


Fig. 11. Output landscape of SELU.



Fig. 13. Output landscape of Swish.

is a smooth non-monotonic activation function and similar to ReLU, it is bounded below and unbounded above. In [31] Mirco Milletari et al. shows that Swish allows more consistent performance over a wide range of the network. The Swish activation function can be expressed as follows:

$$\text{Swish}(x) = \frac{x}{1 + e^{-x}} \quad (11)$$

H. Mish

Mish is a new activation function that has a similar shape and property as Swish. As shown in the figure 16 the main difference is in the concave part of the function. Surprisingly, it provides constant improvement over ReLU and Swish at the cost of more computation. The Mish [11] activation function can be expressed as follows:

$$\text{Mish}(x) = x \tanh(\log(1 + e^x)) \quad (12)$$

V.

VI. CONCLUSION

In this paper, we briefly explained the 2 classes of the activation function. Then we reviewed 8 major activation functions. We see that according to our output landscape SELU, Swish and Mish provide a more smooth output. Therefore those are theoretically easier to optimize. We observe that automated search as in [10] provides a satisfying result for the search of the activation function. Implying that despite the

computational cost of the automated search, this method could provide a satisfying answer across multiple tasks in the future.

REFERENCES

- [1] Y. Qu and M.-X. Wang, "Approximation capabilities of neural networks on unbounded domains," *arXiv:1910.09293 [cs, stat]*, Nov. 2019. arXiv: 1910.09293.
- [2] C. K. Chui and X. Li, "Approximation by ridge functions and neural networks with one hidden layer," *Journal of Approximation Theory*, vol. 70, pp. 131–141, Aug. 1992.
- [3] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals, and Systems*, vol. 2, pp. 303–314, Dec. 1989.
- [4] K.-I. Funahashi, "On the approximate realization of continuous mappings by neural networks," *Neural Networks*, vol. 2, pp. 183–192, Jan. 1989.
- [5] X. Glorot, A. Bordes, and Y. Bengio, "Deep Sparse Rectifier Neural Networks," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pp. 315–323, June 2011.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," *arXiv:1502.01852 [cs]*, Feb. 2015. arXiv: 1502.01852.
- [7] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)," *arXiv:1511.07289 [cs]*, Feb. 2016. arXiv: 1511.07289.
- [8] L. Trottier, P. Giguère, and B. Chaib-draa, "Parametric Exponential Linear Unit for Deep Convolutional Neural Networks," *arXiv:1605.09332 [cs]*, Jan. 2018. arXiv: 1605.09332.
- [9] S. K. Roy, S. Manna, S. R. Dubey, and B. B. Chaudhuri, "LiSHT: Non-Parametric Linearly Scaled Hyperbolic Tangent Activation Function for Neural Networks," *arXiv:1901.05894 [cs]*, Dec. 2018. arXiv: 1901.05894.
- [10] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for Activation Functions," *arXiv:1710.05941 [cs]*, Oct. 2017. arXiv: 1710.05941.
- [11] D. Misra, "Mish: A Self Regularized Non-Monotonic Neural Activation Function," *arXiv:1908.08681 [cs, stat]*, Oct. 2019. arXiv: 1908.08681.

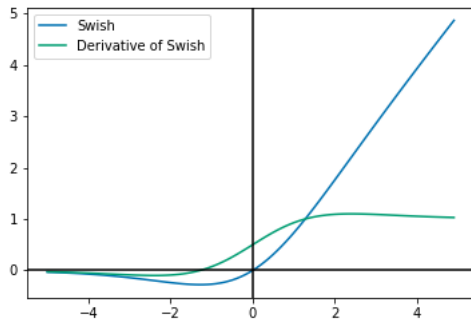


Fig. 14. Plot of the Swish function and its derivative.

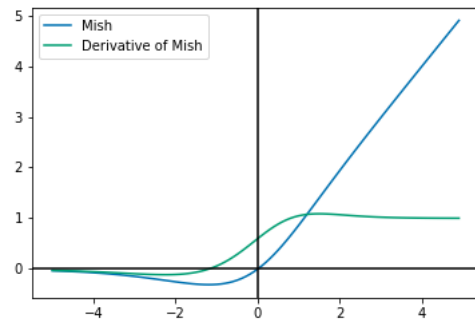


Fig. 16. Plot of the Mish function and its derivative.

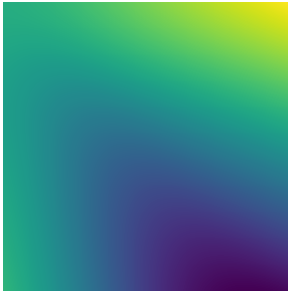


Fig. 15. Output landscape of Mish.

- [12] Alex Krizhevsky, "Convolutional Deep Belief Networks on CIFAR-10." *arXiv:1910.07476 [cond-mat, stat]*, Oct. 2019. arXiv: 1910.07476.
- [13] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter, "Self-Normalizing Neural Networks," *arXiv:1706.02515 [cs, stat]*, Sept. 2017. arXiv: 1706.02515.
- [14] J. Pennington, S. S. Schoenholz, and S. Ganguli, "Resurrecting the sigmoid in deep learning through dynamical isometry: theory and practice," *arXiv:1711.04735 [cs, stat]*, Nov. 2017. arXiv: 1711.04735.
- [15] M. Tanaka, "Weighted Sigmoid Gate Unit for an Activation Function of Deep Neural Network," *arXiv:1810.01829 [cs]*, Oct. 2018. arXiv: 1810.01829.
- [16] A. F. Agarap, "Deep Learning using Rectified Linear Units (ReLU)," *arXiv:1803.08375 [cs, stat]*, Feb. 2019. arXiv: 1803.08375.
- [17] L. Lu, Y. Shin, Y. Su, and G. E. Karniadakis, "Dying ReLU and Initialization: Theory and Numerical Examples," *arXiv:1903.06733 [cs, math, stat]*, Nov. 2019. arXiv: 1903.06733.
- [18] D. Arpit and Y. Bengio, "The Benefits of Over-parameterization at Initialization in Deep ReLU Networks," *arXiv:1901.03611 [cs, stat]*, Oct. 2019. arXiv: 1901.03611.
- [19] J. Berner, D. Elbrächter, and P. Grohs, "How degenerate is the parametrization of neural networks with the ReLU activation function?," *arXiv:1905.09803 [cs, math, stat]*, Oct. 2019. arXiv: 1905.09803.
- [20] A. Dereventsov, A. Petrosyan, and C. Webster, "Neural network integral representations with the ReLU activation function," *arXiv:1910.02743 [cs, stat]*, Oct. 2019. arXiv: 1910.02743.
- [21] S. C. Douglas and J. Yu, "Why ReLU Units Sometimes Die: Analysis of Single-Unit Error Backpropagation in Neural Networks," *arXiv:1812.05981 [cs, eess, stat]*, Dec. 2018. arXiv: 1812.05981.
- [22] J. Ferlez and Y. Shoukry, "AREN: Assured ReLU NN Architecture for Model Predictive Control of LTI Systems," *arXiv:1911.01608 [cs, eess, math]*, Nov. 2019. arXiv: 1911.01608.
- [23] S. Hayou, A. Doucet, and J. Rousseau, "On the Impact of the Activation Function on Deep Neural Networks Training," *arXiv:1902.06853 [cs, stat]*, May 2019. arXiv: 1902.06853.
- [24] S. Goel, S. Karmalkar, and A. Klivans, "Time/Accuracy Trade-offs for Learning a ReLU with respect to Gaussian Marginals," *arXiv:1911.01462 [cs, stat]*, Nov. 2019. arXiv: 1911.01462.
- [25] E. Oostwal, M. Straat, and M. Biehl, "Hidden Unit Specializa-

tion in Layered Neural Networks: ReLU vs. Sigmoidal Activation," *arXiv:1910.07476 [cond-mat, stat]*, Oct. 2019. arXiv: 1910.07476.

- [26] C. Yun, S. Sra, and A. Jadbabaie, "Small ReLU networks are powerful memorizers: a tight analysis of memorization capacity," *arXiv:1810.07770 [cs, stat]*, Oct. 2019. arXiv: 1810.07770.
- [27] T. Wang, Z. Qin, and M. Zhu, "An ELU Network with Total Variation for Image Denoising," *arXiv:1708.04317 [cs]*, Aug. 2017. arXiv: 1708.04317.
- [28] A. Shah, E. Kadam, H. Shah, S. Shinde, and S. Shingade, "Deep Residual Networks with Exponential Linear Unit," *Proceedings of the Third International Symposium on Computer Vision and the Internet - VisionNet'16*, pp. 59–65, 2016. arXiv: 1604.04112.
- [29] Y. Li, C. Fan, Y. Li, Q. Wu, and Y. Ming, "Improving deep neural network with Multiple Parametric Exponential Linear Units," *Neuro-computing*, vol. 301, pp. 11–24, Aug. 2018.
- [30] Z. Huang, T. Ng, L. Liu, H. Mason, X. Zhuang, and D. Liu, "SNDCNN: Self-normalizing deep CNNs with scaled exponential linear units for speech recognition," *arXiv:1910.01992 [cs, eess, stat]*, Oct. 2019. arXiv: 1910.01992.
- [31] M. Milletari, T. Chotibut, and P. E. Trevisanutto, "Mean Field Theory of Activation Functions in Deep Neural Networks," *arXiv:1805.08786 [cs, stat]*, June 2019. arXiv: 1805.08786.